

A Short Guide to Exploratory Data Analysis

Statistical Programming in R

DATA 412/612

Department of Mathematics and Statistics

American University

Fall 2026

What EDA is

Exploratory data analysis is how you get to know a table: ask a question, look at the data, then ask a better question. It is not a dump of every `ggplot` you tried, and it is not a final confirmatory study. You are allowed to be surprised.

R for Data Science puts it this way: two questions come up over and over.

- What kind of variation is there *within* a variable?
- What kind of covariation is there *between* variables?

That is why the project asks for real data with at least three numeric columns, plus categories, dates, and some text.

A reasonable order

You will jump backward when a plot shows a cleaning problem. That is normal. For the write-up, try this order.

1. **One row.** Read the codebook. What is the observational unit?
2. **Import.** `readr` / `readxl` / `haven`, then `glimpse()` and a few `count()`s. Fix types (numbers or dates stored as text are common).
3. **Tidy and clean.** Rename, parse dates, recode labels, join or pivot, drop impossible values. Write down what you dropped.
4. **Missingness.** How much, and does it depend on year or group?
5. **One variable at a time.** Histograms or boxplots for numeric variables; bar charts for categories (reorder the levels); counts over time for dates.
6. **Two or three variables.** Scatterplots, boxplots by group, line plots over time. Facet or color for a third variable. If two numeric variables do not line up in a straight cloud, see below.
7. **Odd points.** Outliers, zeros, and codes like 999 are part of the data. See if your pattern still holds without them.
8. **New questions.** The data-driven hypotheses should not be a copy of the initial ones.
9. **A story.** Keep the plots that you can explain. The rest can go in an appendix or be deleted. The knitted PDF is at most 20 pages.

Type → plot → package

You have	Look at	Tools
one numeric variable	histogram, boxplot, quantiles	ggplot2, summarise
numeric vs. numeric	scatterplot first; smooth / log / bins if it is not a line	geom_point, geom_smooth
numeric vs. category	boxplots, grouped summaries	group_by, forcats
category vs. category	counts, bar charts of proportions	count, geom_bar
dates	parse, then counts or means over time	lubridate
text	flags, splits, cleaned labels, then plot	stringr
many files / groups	the same pipeline repeatedly	purrr
related tables or wide data	joins or pivot_longer	dplyr, tidyr

Reorder factors with `fct_reorder()`, lump rare levels with `fct_lump()`, and put units on the axes. Captions should say something, not “Plot 3.”

When two numeric variables are not a straight line

Pearson correlation and a fitted straight line only describe a *linear* association. They can be near zero when there is a clear curve, and they can look large when the cloud is just two groups sitting apart. Always start with a scatterplot.

- Add a smooth: `geom_smooth()` (loess for smaller n ; for large n use `method = "gam"` or bin first). Read the curve as a description of the cloud, not as a causal model.
- Try a log scale on x or y when the pattern bends or the spread grows with the mean. If the plot is closer to a line on that scale, summarize there and say so.
- Cut one variable into bins (`cut()` or `ntile()`) and plot the mean or median of the other against the bin. That is a simple check that does not assume a line.
- Spearman correlation is for a monotone pattern (always up, or always down) that is not linear. Report it *with* the plot, not instead of the plot.
- If you fit `lm()`, look at residuals vs. fitted. A curve there means the line missed the relationship.
- Facet or color by a categorical variable. A bend in the pooled scatter can be two groups with different intercepts; a straight pooled cloud can hide opposite slopes.

Do this before you write that two variables are related or that they are not. Overplotting can hide a curve; use `alpha` or `geom_hex()` when n is large. The smooth is still exploratory: say what you see, how large it is, and where it might be an artifact of missingness, outliers, or a third variable.

A small pipeline

```
1 library(tidyverse)
2 library(lubridate)
3
4 raw <- read_csv("../data/listings.csv")
5
6 clean <- raw %>%
```

```

7 mutate(
8   price = parse_number(price),
9   last_review = ymd(last_review),
10  neighborhood = fct_lump(neighborhood, n = 10),
11  has_parking = str_detect(amenities, "parking")
12 ) %>%
13 filter(price > 0, price < 2000)
14
15 clean %>%
16   group_by(neighborhood) %>%
17   summarise(n = n(), median_price = median(price, na.rm = TRUE),
18             .groups = "drop") %>%
19   mutate(neighborhood = fct_reorder(neighborhood, median_price)) %>%
20   ggplot(aes(median_price, neighborhood)) +
21   geom_col() +
22   labs(x = "Median_nightly_price_(USD)", y = NULL)

```

Several files of the same shape:

```

1 files <- list.files("../data", pattern = "\\..csv$", full.names = TRUE)
2 trips <- files %>% map_dfr(read_csv, .id = "source")

```

A nonlinear numeric-numeric plot:

```

1 clean %>%
2   ggplot(aes(reviews, price)) +
3   geom_point(alpha = 0.3) +
4   geom_smooth(se = FALSE) +
5   scale_x_log10() +
6   facet_wrap(~ room_type) +
7   labs(x = "Number_of_reviews", y = "Nightly_price_(USD)")

```

Writing it up

For each important figure: one sentence of setup, the figure, then two or three sentences that state the pattern, its size, and a caveat. Connect figures instead of listing them. Do not claim a relationship that the plot does not show, and do not treat a smooth or a correlation as the end of the argument.

A sequence that usually works: structure and missingness; the main numeric variables; the main groups; the relationship you care about (stratified); the same thing over time if you have dates; one surprise.

The EDA chapter in *R for Data Science* is the main reading: <https://r4ds.hadley.nz/eda>.